

Вероятностные алгоритмы проверки чисел на простоту

Филибер Никола

13 окт 2026, Москва, Россия

Российский Университет Дружбы Народов

Цель работы

Изучение и практическое применение методов программной реализации вероятностных алгоритмов проверки чисел на простоту.

Задание

1. Алгоритм, реализующий тест Ферма.
2. Алгоритм вычисления символа Якоби.
3. Алгоритм, реализующий тест Соловья-Штрассена.
4. Алгоритм, реализующий тест Миллера-Рабина.

Теоретическое введение

- Вход. Нечетное целое число $n \geq 5$.
 - Выход. «Число n , вероятно, простое» или «Число n составное».
1. Выбрать случайное целое число a , $2 \leq a \leq n - 2$.
 2. Вычислить $r = a^{n-1} \pmod{n}$
 3. При $r = 1$ результат: «Число n , вероятно, простое». В противном случае результат: «Число n составное»..

Тест Соловья-Штрассена

- Вход. Нечетное целое число $n \geq 5$.
 - Выход. «Число n , вероятно, простое» или «Число n составное».
1. Выбрать случайное целое число a , $2 \leq a \leq n - 2$.
 2. Вычислить $r = a^{(\frac{n-1}{2})} \pmod n$
 3. При $r \neq 1$ и $r \neq n - 1$ результат: «Число n составное».
 4. Вычислить символ Якоби $s = \left(\frac{a}{n}\right)$
 5. При $r = s \pmod n$ результат: «Число n , вероятно, простое». В противном случае результат: «Число n составное».

Тест Миллера-Рабина.

1. Представить $n - 1$ в виде $n - 1 = 2^s r$, где r - нечетное число
2. Выбрать случайное целое число a , $2 \leq a \leq n - 2$.
3. Вычислить $y = a^r \pmod{n}$
4. При $y \neq 1$ и $y \neq n - 1$ выполнить действия
 - Положить $j = 1$
 - Если $j \leq s - 1$ и $y \neq n - 1$ то
 - Положить $y = y^2 \pmod{n}$
 - При $y = 1$ результат: «Число n составное».
 - Положить $j = j + 1$
 - При $y \neq n - 1$ результат: «Число n составное».
5. Результат: «Число n , вероятно, простое».

Выполнение лабораторной работы

Тест Ферма

```
import random

def is_probable_prime_fermat(n: int, rounds: int = 10) -> bool:
    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    for _ in range(rounds):
        a = random.randrange(2, n - 1)
        if gcd(a, n) != 1:
            return False # composite
        if pow(a, n - 1, n) != 1:
            return False # composite
    return True # probably prime
```

Рис. 1: Тест Ферма

Тест Миллера-Рабина

```
def jacobi(a: int, n: int) -> int:
    """
    Compute Jacobi symbol (a/n) for odd n >= 3.
    Returns: -1, 0, or 1.

    This matches the algorithmic rules typically used in Solovay-Strassen:
    - factor out powers of 2 from a and adjust sign by n mod 8
    - apply quadratic reciprocity when swapping (a, n)
    - reduce a mod n
    """
    if n <= 0 or n % 2 == 0:
        raise ValueError("Jacobi symbol is defined here for positive odd n.")

    a %= n
    result = 1

    while a != 0:
        # Step A: remove factors of 2 from a
        while a % 2 == 0:
            a //= 2
            r = n % 8
            if r == 3 or r == 5:
                result = -result

        # Step B: quadratic reciprocity swap
        a, n = n, a # swap

        if (a % 4 == 3) and (n % 4 == 3):
            result = -result

    a %= n

    # if n == 1 => result, else a became 0 with n>1 => gcd != 1 => 0
    return result if n == 1 else 0
```

```
print(jacobi(1001, 9907))
```

```
-1
```

🏠 ⬆ ⬇ ⬅

Тест Соловья-Штрассена

```
def is_probable_prime_solovay_strassen(n: int, rounds: int = 10, rng: Optional[random.Random] = None) -> bool:
    """
    Solovay-Strassen:
    pick random a in [2, n-2]
    if gcd(a, n) > 1 -> composite
    r = a^((n-1)/2) mod n
    s = jacobi(a, n)
    compare r with s (mod n)
    """
    if rng is None:
        rng = random.Random()

    if n < 2:
        return False
    if n in (2, 3):
        return True
    if n % 2 == 0:
        return False

    for _ in range(rounds):
        a = rng.randrange(2, n - 1)
        g = gcd(a, n)
        if g > 1:
            return False

        r = mod_pow(a, (n - 1) // 2, n)
        s = jacobi(a, n)
        if s == 0:
            return False

        # s is -1 or 1; compare modulo n
        s_mod = s % n
        if r != s_mod:
            return False

    return True
```

```
print(is_probable_prime_solovay_strassen(11,10))
```

True

Выводы

Изученил и разработал методы программной реализации алгоритма шифрования гаммированием конечной гаммой.

Список литературы

https://en.wikipedia.org/wiki/Primality_test